GUIA RÁPIDO DE ESTUDO E DEFESA DO PROJETO (EM LINGUAGEM SIMPLES)

PicoClaw (Go) vs. OpenClaw (Node.js/TypeScript) no Apple Silicon M4
Rafael Paniccia Cassetta (RA: 21202520618) — CCM313-EEI203 (UFABC)

1. Qual é a Ideia Central do Projeto? (Explicação para Leigos)

Imagine que uma Inteligência Artificial (o modelo Llama-3.2 de 3 bilhões de parâmetros) é o 'motor' de um carro, e o Agente de IA (PicoClaw ou OpenClaw) é o 'motorista/painel de controle' que recebe seu pedido, abre arquivos no computador, faz contas e conversa com o motor. Se colocarmos exatamente o mesmo motor nos dois carros, qual motorista é mais rápido, gasta menos memória RAM e desperdiça menos bateria?

- **OpenClaw (feito em TypeScript / Node.js):** É como um motorista que carrega uma mochila gigante com 54 ferramentas e precisa de um tradutor em tempo real (a máquina virtual V8 do JavaScript). É muito famoso e cheio de recursos, mas é pesado.
- **PicoClaw (feito em Go / Golang pela Sipeed):** É um motorista atleta, compilado direto para a língua do processador do Mac (código nativo ARM64). Não precisa de tradutor e carrega só o essencial.

2. O Que é Cada Ferramenta que Usamos?

Ferramenta	O que ela é (em linguagem simples)	Papel no nosso teste
OpenClaw (v2026.10.7)	Agente de IA famoso escrito em TypeScript/Node.js (v26.12.1).	Competidor 1 (pesado/modular).
PicoClaw (v0.3.1)	Agente de IA ultraleve escrito em linguagem Go (v1.23.2).	Competidor 2 (leve/compilado).
Llama-3.2-3B (GGUF)	O 'cérebro' de IA da Meta (compactado em 4-bits, 1,92 GB).	Responde às perguntas dos dois agentes de forma idêntica.
llama.cpp (b11887)	O servidor que roda o cérebro de IA usando a placa de vídeo (Metal GPU) do Mac M4.	Garante que os dois agentes usem exatamente a mesma IA local.
hyperfine (v2.0.0)	Um 'cronômetro olímpico' de altíssima precisão.	Mediu o tempo exato de abertura (Cold Start) 30 vezes seguidas.
powermetrics + libproc	Sensores oficiais da Apple dentro do chip M4 (exigem senha de administrador/ root).	Mediram quantos Watts (energia), ciclos de CPU e memória RAM cada agente gastou.
Python 3.12 + uv	Linguagem que automatizou a bateria de testes e gerou os gráficos.	Rodou os 480 testes sorteados e calculou a estatística (t-test).
caffeinate + mdutil	Utilitários do macOS para bloquear hibernação e pausar buscas do Spotlight.	Garantiram que o Mac não dormisse nem fizesse tarefas em segundo plano durante o teste.

3. Tudo o Que Foi Feito no Trabalho (Passo a Passo)

- **Passo 1 (Instalação e Calibração):** Baixamos todas as ferramentas no Mac mini M4 (24 GB de RAM) e colocamos o modelo Llama-3.2-3B para rodar 100% local (sem internet, na porta 127.0.0.1:8080) com temperatura 0.0 (para a IA nunca mudar de resposta por sorte).
- **Passo 2 (As 4 Tarefas / Cargas de Trabalho — W1 a W4):** Criamos 4 desafios de dificuldade crescente para os dois agentes:
  • W1 (Fácil): Apenas responder uma pergunta direta sem usar ferramentas.
  • W2 (Médio): Executar 3 ferramentas seguidas (listar 50 arquivos, filtrar um arquivo de log de 750 linhas e calcular a média/P99).
  • W3 (Pesado - RAG): Buscar os 5 textos mais parecidos dentro de uma base com 10.000 documentos usando vetores matemáticos (384 dimensões).
  • W4 (Estresse): Um ciclo autônomo de 10 rodadas seguidas de raciocínio e uso de ferramentas (ReAct Loop).
- **Passo 3 (Frio vs. Quente — 480 Testes no Total):** Testamos os 2 agentes × 4 tarefas × 2 estados ('Cold Start', abrindo o programa do zero, e 'Warm', com o programa já aberto). Isso dá 16 combinações (T1 a T16). Repetimos cada uma 30 vezes em ordem sorteada = 480 testes no total!

4. Cálculos Matemáticos Usados para Gerar os Gráficos (Com Exemplos Reais)

Se o professor perguntar 'como você calculou o número que está neste gráfico?', aqui estão todas as fórmulas traduzidas para o português, com as contas exatas feitas pelo nosso código:

Nome da Métrica (e Gráfico)	Fórmula Matemática	Significado Simples + Conta Real do Nosso Teste
1. Atraso Puro do Agente (T_orch) [Figura 4a]	T_orch = T_task - T_neural	Tempo Total menos o Tempo da IA. Isola só a lentidão do Agente. • OpenClaw (T1): 1.693,53 - 1.449,07 = 244,46 ms • PicoClaw (T2): 1.460,17 - 1.449,07 = 11,10 ms
2. Fração de Overhead (Φ_orch) [Figura 4b]	Φ_orch = (T_orch / T_task) × 100%	Qual % do tempo total a culpa foi do Agente (e não da IA). • OpenClaw (T1): (244,46 / 1.693,53) × 100 = 14,41% • PicoClaw (T2): (11,10 / 1.460,17) × 100 = 0,76%
3. Latência Entre Palavras (ITL) [Tabela 1 e 2]	ITL = (T_task - TTFT) / (N_out - 1)	Tempo médio entre cada palavra gerada após a 1ª palavra (TTFT). • Ex. (W1, 64 tokens): (1.460,2 - 208,6) / 63 ≈ 19,86 ms/token
4. Vazão de Ferramentas (R_tool) [Figura 6a]	R_tool = N_tools / (T_dispatch / 1000)	Quantas ferramentas o agente consegue processar por segundo. • PicoClaw (W2, 3 tools em 9,16 ms): 3 / 0,00916s = 327,4 ops/s • OpenClaw (W2, 3 tools em 46,58 ms): 3 / 0,04658s = 64,4 ops/s
5. Amplificação de RAM (α_mem) [Figura 1b]	α_mem = RSS_pico / RSS_idle	Quantas vezes a memória RAM cresce no pico em relação ao repouso. • PicoClaw (W4): 33,83 MB / 14,02 MB = 2,41x • OpenClaw (W4): 448,33 MB / 166,10 MB = 2,70x (mas parte de 166 MB!)
6. Energia do Chip em Joules (E_pkg) [Figura 5a e 5b]	E_pkg = ∑ [P_Watts(t) × Δt_seg]	Potência (Watts) multiplicada pelo tempo (segundos). A cada 0,1s do powermetrics, somamos (Watts × 0,1s). • PicoClaw (W1): ~14,83 W × 1,460 s = 21,65 Joules • OpenClaw (W1): ~14,45 W × 1,694 s = 24,47 Joules
7. Produto Energia-Atraso (EDP) [Figura 5c]	EDP = E_pkg × (T_task / 1000)	Multiplica Energia × Tempo (J·s). Penaliza quem gasta mais energia E demora mais. • PicoClaw (W1): 21,65 J × 1,460 s = 31,62 J·s • OpenClaw (W1): 24,47 J × 1,694 s = 41,48 J·s
8. Projeção RAG 100k + Cosseno [Tier W3]	v_q = x·W + b (98.688 params) cos(θ) = (v_q · d_i) / (‖v_q‖ ‖d_i‖)	Multiplica o vetor de entrada (256) pela matriz (256×384 + 384 bias = 98.688 parâmetros ≈ 100k) e compara com 10.000 textos.
9. Intervalo de Confiança (IC 95%) [Barras de erro nos gráficos]	IC_95% = Média ± 2,045 × (DP / √30)	São as 'anteninhas' no topo das barras dos gráficos. Como fizemos n=30 testes, usamos t=2,045 (Student, 29 graus de liberdade). Garante 95% de certeza do valor real.

10. Teste t de Welch (t e p-value) [Tabela 3 do Relatório]	t = (Média_Open - Média_Pico) / √(DP_Open²/30 + DP_Pico²/30)	Compara se um agente é realmente diferente do outro. Como p-value deu menor que 0,01 (ex: 3,25 × 10^-23), provamos com >99% de certeza que não foi sorte!

5. Conclusões Principais (Os 5 Números de Ouro para Você Lembrar)

O que medimos	PicoClaw (Go)	OpenClaw (Node.js)	Conclusão Simples
1. Tempo para abrir (Cold Start)	9,6 ms	225,6 ms	O PicoClaw abre 23,4 vezes mais rápido (quase instantâneo).
2. Consumo de Memória RAM	14 MB (parado) a 34 MB (pico)	166 MB (parado) a 448 MB (pico)	O OpenClaw gasta de 12x a 13x mais memória RAM.
3. Trabalho do Processador ao abrir	156 milhões de instruções	4.650 milhões de instruções	O Node.js faz o processador trabalhar 30x mais só para ligar.
4. Velocidade das Ferramentas	327 ferramentas/seg	64 ferramentas/seg	O PicoClaw processa ferramentas 5,1 vezes mais rápido.
5. Energia da Placa de Vídeo (GPU)	~15,64 Watts (21,6 J no W1)	~15,72 Watts (24,5 J no W1)	A placa de vídeo (11,6 W) domina o gasto quando o texto é igual.

💡 A Grande Descoberta Prática do System Testing: Quando usamos o OpenClaw do jeito que ele vem de fábrica (sem equalizar o prompt), ele injeta um texto oculto de 40.799 caracteres (16.369 tokens, com 54 ferramentas) antes mesmo de enviar um simples 'Oi'. Isso fez o limite original de 4.096 tokens travar com erro (context_overflow) e obrigou a placa de vídeo a trabalhar por 57 segundos gastando ~560 Joules de energia só para ler o manual de ferramentas do OpenClaw!

6. Perguntas que o Professor Pode Fazer e Como Você Deve Responder

> ❓ **Pergunta do Professor: O que significam as barras de erro ('anteninhas') nos gráficos e por que alguns eixos estão em escala logarítmica (log scale)?**
>
> ✅ **Como Responder:** As barras de erro representam o Intervalo de Confiança de 95% (calculado com a distribuição t de Student para n = 30 repetições e 29 graus de liberdade: Média ± 2,045 × DesvioPadrão/√30). E usamos escala logarítmica (potências de 10) nos gráficos de instruções de CPU e EDP porque a diferença entre o OpenClaw (4.650 milhões de instruções) e o PicoClaw (156 milhões) é de quase 30 vezes — numa escala comum, a barra do PicoClaw ficaria quase invisível!

> ❓ **Pergunta do Professor: Por que você precisou fazer ajustes em relação ao Work Plan original (Seção 6)?**
>
> ✅ **Como Responder:** Professor, durante o System Testing encontramos 3 barreiras reais que todo teste empírico encontra: (1) A versão atual do OpenClaw (v2026.10.7) usa o banco nativo node:sqlite e exige Node.js 26.12+, bloqueando o Node 22 LTS; (2) O OpenClaw injeta por padrão 54 ferramentas (16.369 tokens) no prompt, o que estourava o limite de 4.096 tokens do Work Plan. Por isso aumentamos a janela do servidor para 32.768 tokens para testar a CLI padrão e criamos workers equalizados para os 480 testes; (3) Usamos permissão de superusuário (root) para capturar as curvas reais de potência no powermetrics e pausar o Spotlight (mdutil), combinando com os contadores de hardware RUSAGE_INFO_V4 do kernel Darwin.

❓ **Pergunta do Professor: Como você separou o tempo gasto pela IA (Llama-3.2) do tempo gasto pelo Agente (PicoClaw/OpenClaw)?**

✅ **Como Responder:** O servidor llama.cpp nos devolve no JSON de resposta o tempo exato em milissegundos que a placa de vídeo levou para processar o prompt e gerar os tokens (chamamos isso de T_neural). Nós medimos o tempo total de ponta a ponta (T_task) com relógio monotônico de nanossegundos e subtraímos um do outro (T_orch = T_task - T_neural). Assim isolamos puramente o atraso do Go contra o atraso do Node.js.

❓ **Pergunta do Professor: Se o PicoClaw é 22 vezes mais rápido na orquestração, por que o consumo de energia (Joules) nos testes equalizados ficou parecido?**

✅ **Como Responder:** Porque no Apple M4, conforme medimos com o sudo powermetrics, a GPU Metal rodando o modelo de IA consome sozinha cerca de 11,6 Watts (de um total de ~15,7 Watts do chip inteiro), e como o modelo de IA demora de 1,4s a 22s para gerar o texto, a GPU domina a conta de energia quando os prompts têm o mesmo tamanho. Porém, no uso real de fábrica, como o OpenClaw envia 16.369 tokens de sistema a cada conversa, ele faz a GPU gastar mais de 500 Joules extras!

❓ **Pergunta do Professor: Por que você não desligou o Wi-Fi do Mac mini como estava escrito no Work Plan original?**

✅ **Como Responder:** Porque o ambiente de automação e controle dos testes precisava da conexão ativa. Para garantir que o Wi-Fi não interferisse em nada nos testes, amarramos toda a comunicação dos agentes com o servidor de IA estritamente ao endereço local de loopback (127.0.0.1:8080), que não passa pela placa de rede física, além de pausar a indexação do Spotlight com sudo mdutil -a -i off e bloquear a hibernação com caffeinate.

❓ **Pergunta do Professor: Qual é a sua recomendação final de engenharia com base nos dados?**

✅ **Como Responder:** Se o objetivo for rodar um agente sempre ativo em segundo plano no computador do usuário, em dispositivos de borda (Raspberry Pi / Mac com pouca RAM) ou chamadas rápidas via linha de comando, o PicoClaw (Go) é amplamente superior: abre em 9,6 ms e gasta apenas 14–34 MB de RAM. O OpenClaw (Node.js) só se justifica se o desenvolvedor depender especificamente de algum plugin pronto do ecossistema NPM e tiver memória RAM de sobra (>500 MB).